

Week 1: Introduction to Data Engineering

Course Logistics

- 2 weekly hours
- 15% assignments
- 85% final exam
- or.perets@shenkar.ac.il

Prerequisites

- SQL query design and implementation
- Basic data structures
- Complexity and running-time intuition
- Probability

Learning Outcomes

- Core **data engineering** constraints
- Design a basic source-to-consumer pipeline
- Data engineer skills: DWH design, maintenance, ETL
- Storage and performance optimization

What Is Big Data?

- Data that exceeds traditional single-system processing
- Defined by scale, speed, and complexity
- Not a single tool or product

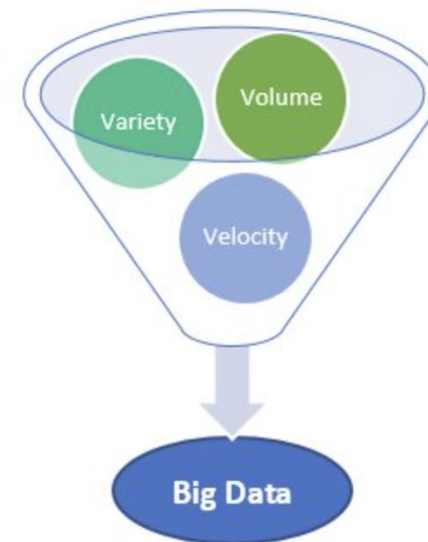


Big Data in Practice

- User behavior tracking
- Mobile and web application logs
- IoT and sensor streams
- Media and social network content

The 3Vs of Big Data

- **Velocity:** data arrives continuously and quickly
- **Variety:** structured, semi-structured, unstructured
- **Volume:** very large scale (TB to PB)



Cloud Service Models

- **SaaS:** ready-to-use applications
- **PaaS:** managed platform for development
- **IaaS:** virtual compute, storage, networking



Data Scientist vs Data Engineer

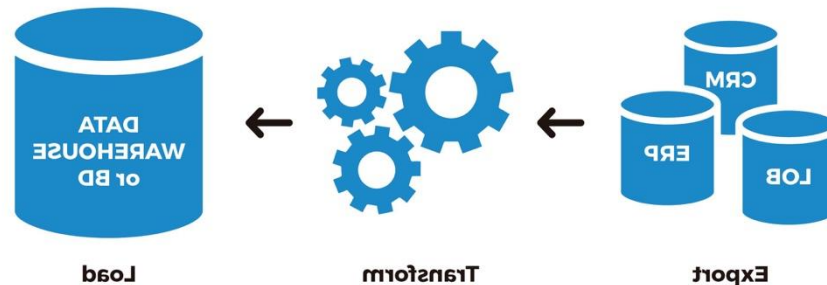
- Data scientists create models and insights
- Data engineers build reliable data platforms
- Both roles are interdependent in production systems

Data Engineer: Core Responsibility

- Build and maintain the data platform
- Ensure reliable ingestion and transformation
- Deliver trusted datasets for analytics and ML

What Is Data Engineering?

- Design systems that move and transform data
- Serve dependable data for analytics, apps, and ML
- Optimize for correctness, latency, and stability



Data Engineering Is Constraint Management

- Sources are fragmented and heterogeneous
- Data volume outgrows single-node capacity
- Pipelines fail and must recover safely
- Cost, latency, and reliability must be balanced

Core Problem Statement

- Many producers, one consistent data view
- Fresh data with clear quality guarantees
- Scalable architecture without uncontrolled costs
- Repeatable pipelines

Why Single Scripts Break

- Runtime grows with data size
- Limited observability slows incident response
- Missing raw layer blocks safe replay
- Non-idempotent writes corrupt outputs on rerun

Storage Design

- **Partition** by date or high-value keys
- **Replicate** for durability and availability
- **Model** hot vs cold storage costs explicitly

E-Commerce Clickstream

- Input: 20M events/day
- Goal: daily product page-view aggregates
- Data need to be available within 1 hour
- Reliability target: 99.9% successful runs

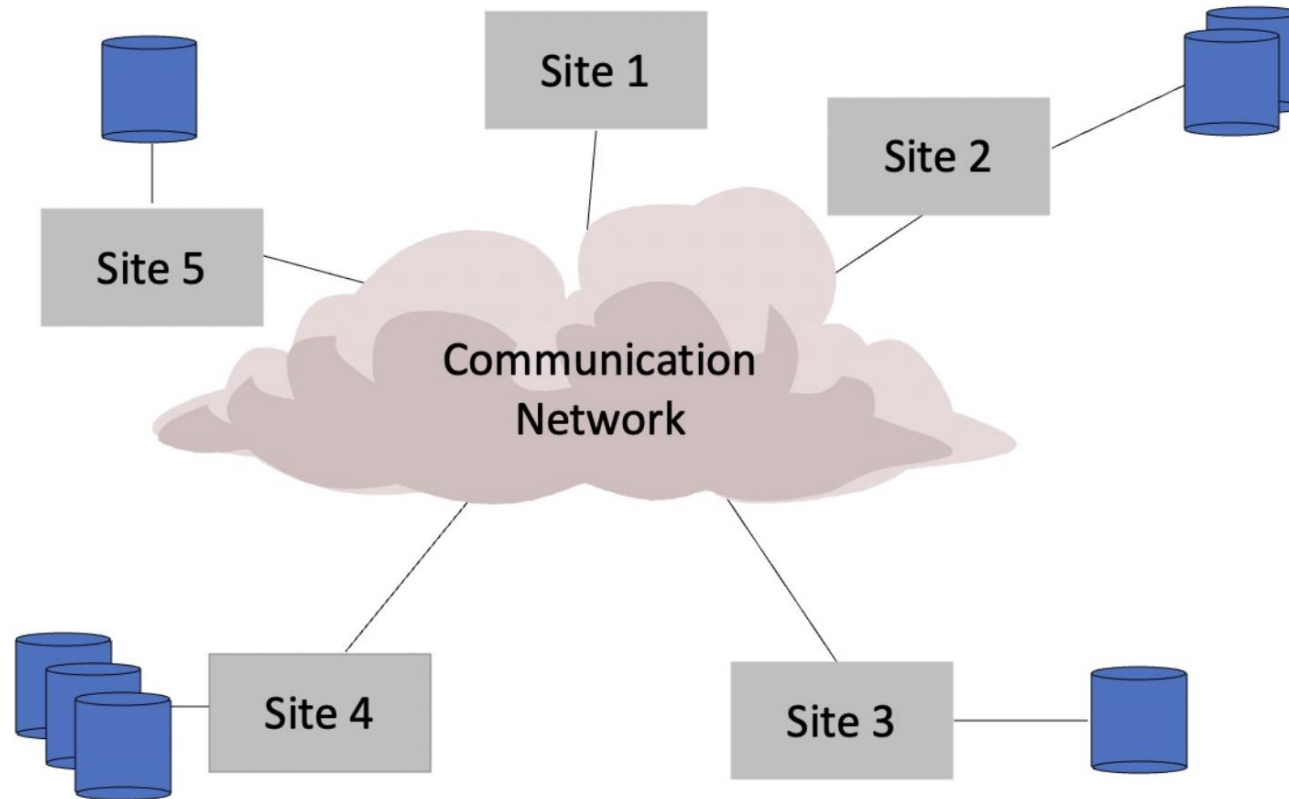
Good luck for “regular” techniques....

Course Overview: SQL Optimization

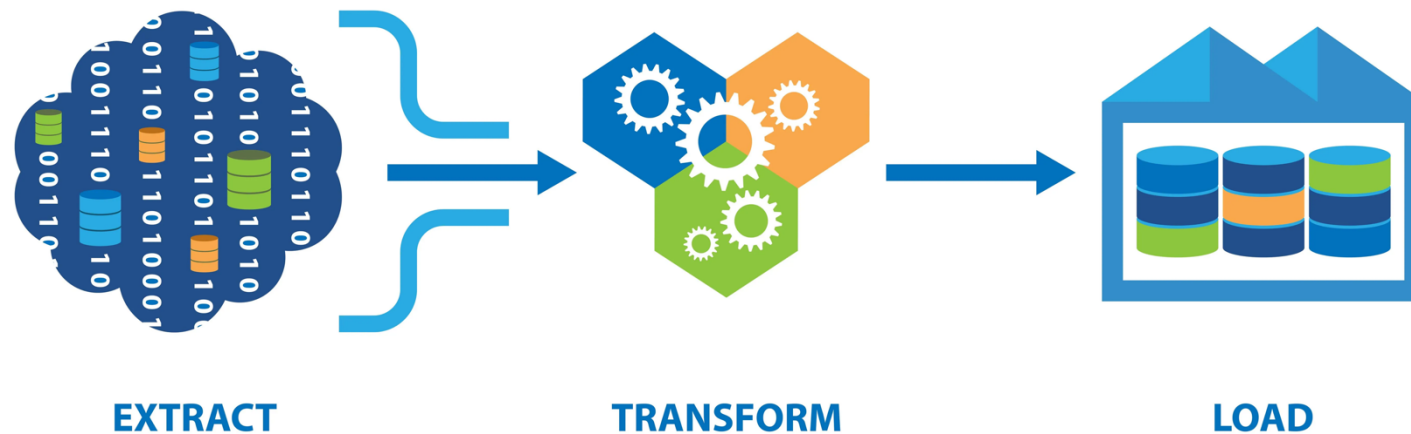
- Window functions
- CTEs
- Sub-queries



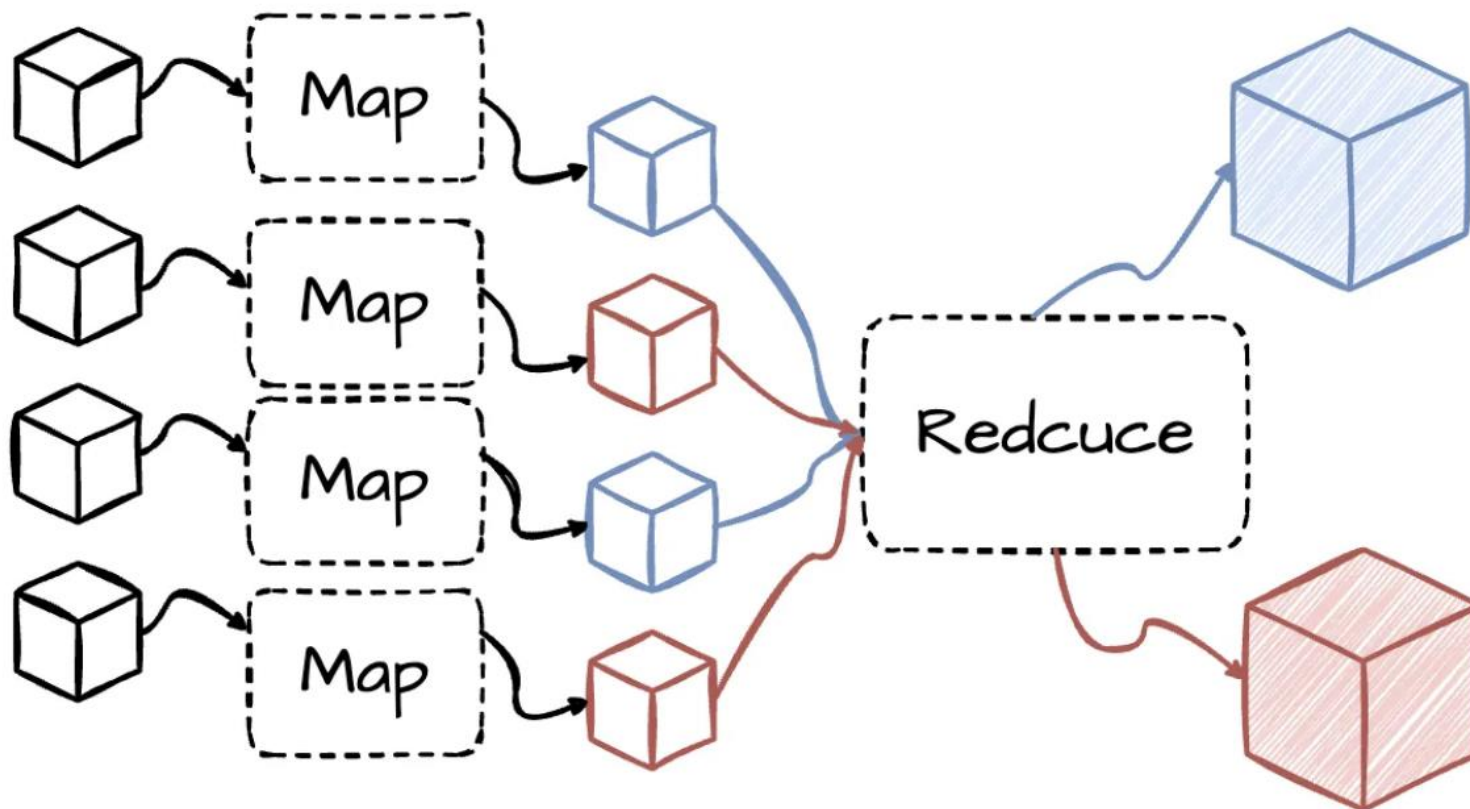
Course Overview: Distributed Computing



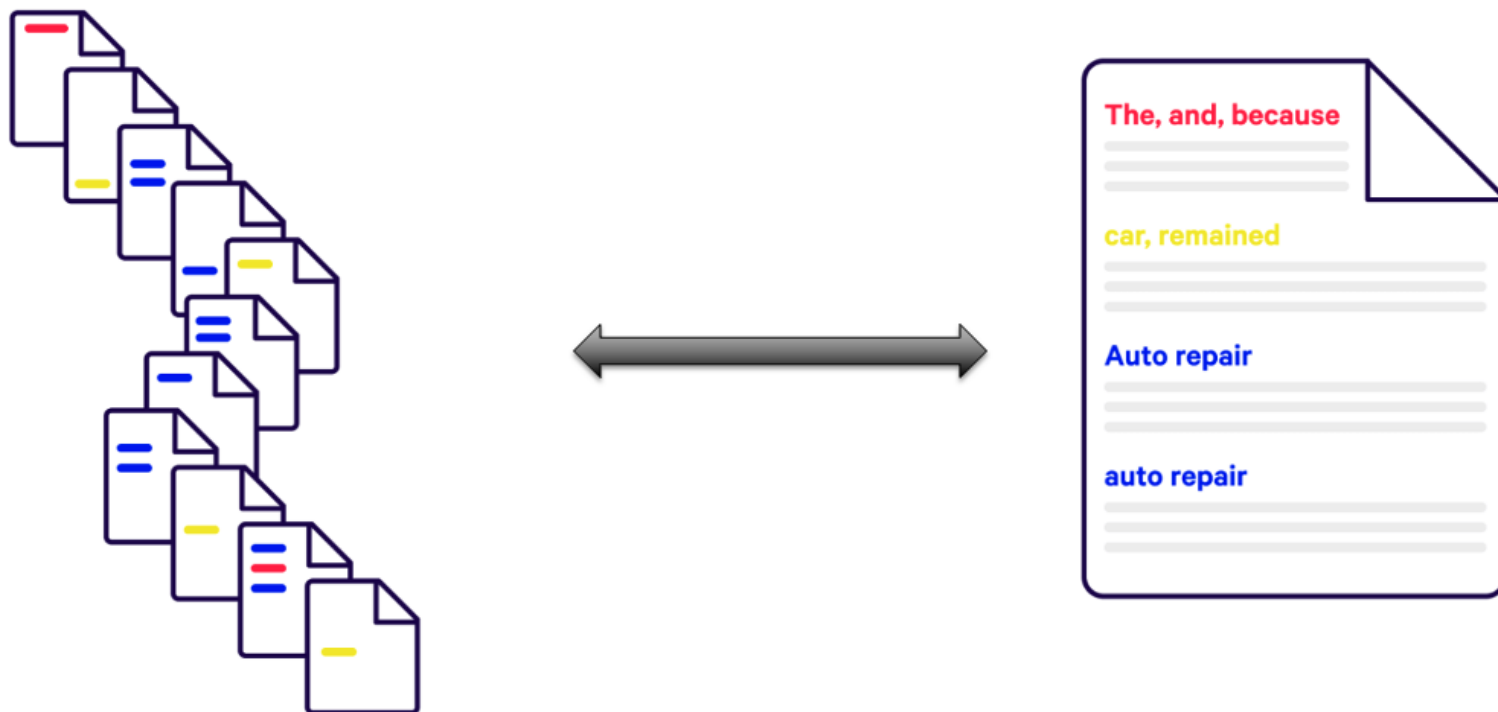
Course Overview: Data Warehouse, Data Lake, ETL



Course Overview: Map Reduce



Course Overview: TF-IDF



Course Overview: Text Processing for LLMs

Course Overview: Streaming Data

SQL – Recap

Recap SQL

- Differentiate SQL DML vs. DDL and apply each correctly
- Choose the right join type and understand NULL results
- Use subqueries with `ALL`, `IN`, and `EXISTS`
- Apply window functions for aggregation, ranking, and value navigation

SQL Foundations (DML vs. DDL)

- **SQL** is a query language that is table-oriented and optimized for fast access to data.
- **CRUD** operations: Create, Read, Update, Delete.
- **DML**: SELECT, INSERT, UPDATE, DELETE
- **DDL**: CREATE, DROP, ALTER

Create + Insert + Verify (Example)

```
CREATE TABLE Students (  
    Id int NOT NULL,  
    First_Name text,  
    Last_Name text,  
    Birth_Date date,  
    Hour_Salary float  
);
```

```
INSERT INTO Students  
VALUES (123456789, 'Israel', 'Israeli', '1980-03-20', 120.40);
```

Example: Students and Courses Tables

FirstName	LastName	StudentID
Avi	Cohen	111
Dan	Israeli	222
Ofer	Bar	333

StudentID	CourseNumber	CourseName	Grade
111	289	DB	96
111	281	Algo	85
222	281	Algo	78

Primary Key: StudentID in Students, **Foreign Key:** StudentID in Courses

How to Merge Two Tables? - Option 1

```
SELECT FirstName, CourseName, Grade  
FROM Students AS S, Courses AS C  
WHERE (S.StudentID = C.StudentID)  
      AND (C.CourseName = 'Algo');
```

Option 2 (Explicit JOIN)

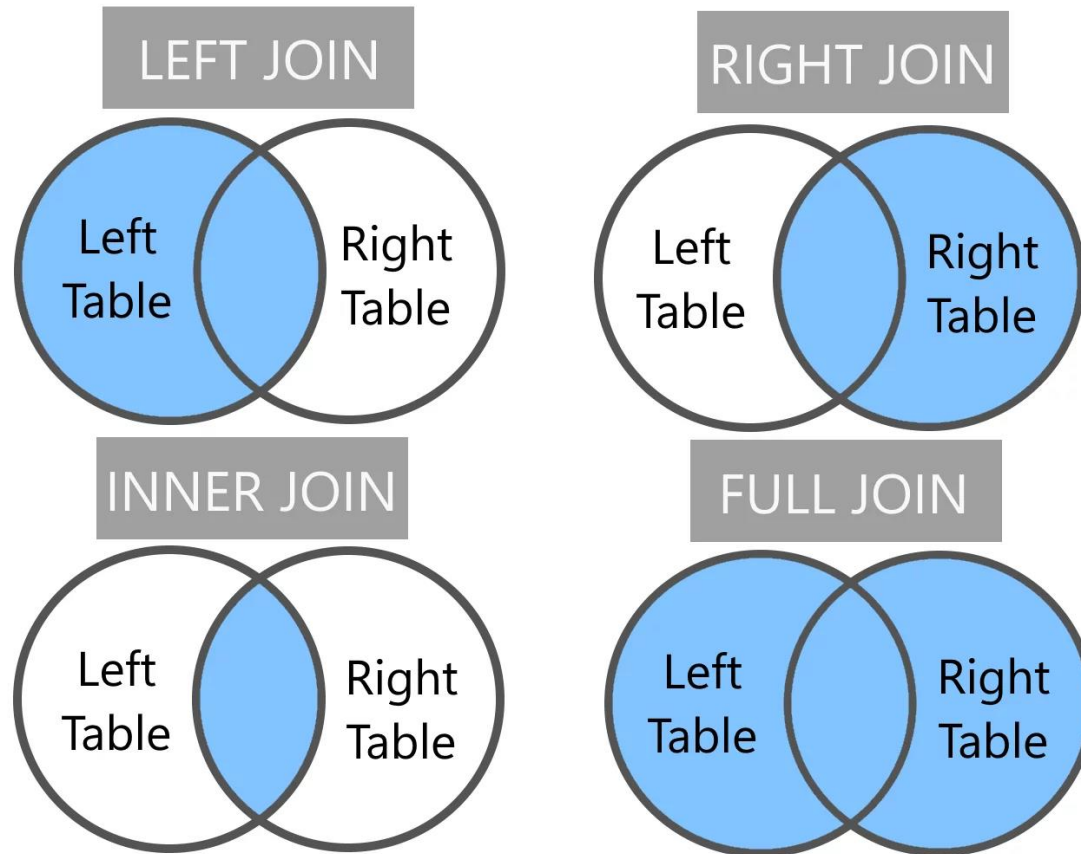
```
SELECT FirstName, CourseName, Grade
FROM Students AS S JOIN Courses AS C
    ON S.StudentID = C.StudentID
WHERE C.CourseName = 'Algo';
```

ON vs. USING

- Use **ON** when join column names differ.
- Use **USING** when column names are the same.

```
SELECT FirstName, CourseName, Grade  
FROM Students  
JOIN Courses USING (StudentID)  
WHERE CourseName = 'Algo';
```

Join Types



Handling NULL Values

- NULL values can come from:
 - Left/Right/Full outer joins
 - Missing data during inserts
 - Other data quality issues

Example: Find Rows with Missing Course Data

```
SELECT FirstName, LastName, CourseNumber  
FROM Student LEFT OUTER JOIN Courses  
USING (StudentID)  
WHERE CourseNumber IS NULL
```

- Use `IS NULL / IS NOT NULL`.
- Wrong usage: `FirstName = 'NULL'`
NULL is not a string.

Subqueries: ALL

Find the city whose average amount is greater than or equal to all city averages:

```
SELECT S.FirstName
FROM Students S JOIN Courses C
ON S.StudentID = C.StudentID
GROUP BY S.StudentID
HAVING AVG(C.Grade) >= ALL (
    SELECT AVG(Grade)
    FROM Courses
    GROUP BY StudentID
);
```

Subqueries: IN

Find all names living in specific cities:

```
SELECT LastName
```

```
FROM Students
```

```
WHERE FirstName IN ( 'AVi', 'Ofer' );
```

Subqueries: EXISTS

```
SELECT S.FirstName  
FROM Students S  
WHERE EXISTS (  
    SELECT *  
    FROM Courses C  
    WHERE C.StudentID = S.StudentID  
);
```